

# Azure, ADF, Databricks & Code Industrialization — Reference

EY Probation Training — Week 4 (FEU Technical Architecture). Covers Azure infrastructure, ADF & Databricks, Azure DevOps & code industrialization, backtesting and parallel run.

Week 4 mixes **generic platform knowledge** (in this document) with **FEU-specific configuration** (only obtainable from their KT sessions and SharePoint). Learn the generic layer here so that in the sessions you're mapping their choices onto concepts you already hold, rather than meeting both at once.

## 1. Azure Data Platform Landscape

| Service                       | Role                                                                |
|-------------------------------|---------------------------------------------------------------------|
| ADLS Gen2                     | Hierarchical data lake storage; the usual landing and curated zones |
| Azure Data Factory (ADF)      | Orchestration and ELT — pipelines, triggers, copy activities        |
| Azure Databricks              | Spark compute; notebooks, jobs, Delta Lake                          |
| Azure Synapse                 | Analytics warehouse; SQL pools                                      |
| Azure Key Vault               | Secrets, keys, certificates                                         |
| Azure DevOps                  | Repos, Pipelines, Boards, Artifacts                                 |
| Azure Monitor / Log Analytics | Metrics, logs, alerting                                             |
| Microsoft Entra ID            | Identity (formerly Azure AD)                                        |

### Medallion architecture

The near-universal lakehouse layout, and almost certainly how FEU is organised:

- **Bronze** — raw ingested data, append-only, schema-on-read, full history. Never edited.
- **Silver** — cleaned, deduplicated, conformed, joined. Business entities.
- **Gold** — aggregated, business-level tables and feature sets ready for consumption.

The value is reprocessability: if Silver logic is wrong, you rebuild it from Bronze without re-ingesting from source.

### Identity and access

- **Service Principal** — an identity for an application, used for non-interactive pipeline auth.
- **Managed Identity** — an Azure-managed service principal with no credentials to store. Preferred where supported.
- **RBAC** — role assignments at subscription/resource-group/resource scope.
- **Key Vault** — never hard-code secrets. ADF and Databricks both read from Key Vault; in Databricks it appears as a secret scope.

## 2. Azure Data Factory

### Core concepts

| Concept                  | Meaning                                                          |
|--------------------------|------------------------------------------------------------------|
| Pipeline                 | A container of activities, forming a workflow                    |
| Activity                 | One step — Copy, Notebook, Stored Procedure, Lookup, ForEach, If |
| Dataset                  | A named view of the data a activity reads or writes              |
| Linked Service           | The connection string / credential to a data store or compute    |
| Integration Runtime (IR) | The compute that runs the activity                               |
| Trigger                  | What starts a pipeline                                           |

### Integration Runtime types

- **Azure IR** — fully managed, for cloud-to-cloud movement and data flows.
- **Self-hosted IR** — installed on a VM or on-premises machine, for accessing private networks and on-prem sources.
- **Azure-SSIS IR** — lift-and-shift for existing SSIS packages.

### Trigger types

- **Schedule** — wall-clock recurrence.
- **Tumbling window** — fixed-size, non-overlapping, contiguous time slices, with dependency and backfill support. This is what you want for reliable incremental loads.
- **Event-based** — fires on blob created/deleted.
- **Manual / on-demand**.

Schedule vs tumbling window is a likely question: tumbling windows retain state, can be re-run for a historical slice, and support dependencies between windows; schedule triggers are fire-and-forget.

## Control flow activities

`ForEach` (with an optional `batchCount` for parallelism), `If Condition`, `Switch`, `Until`, `Lookup` (retrieve a config row or list), `Get Metadata`, `Execute Pipeline`, `Web`, `Wait`, `Fail`, `Set/Append Variable`.

**Parameterisation** — the difference between a maintainable factory and 200 near-identical pipelines. Parameterise datasets and linked services, drive a `ForEach` from a metadata/config table, and pass parameters through `@pipeline().parameters.x`. Expressions use the ADF expression language: `@concat()`, `@utcnow()`, `@formatDateTime()`, `@activity('X').output`.

## Mapping Data Flows

Visual, code-free transformations that compile to Spark and run on the Azure IR. Useful for teams without Spark skills; most engineering teams prefer Databricks notebooks called from ADF, because notebooks are versionable, testable and reviewable as code.

## Monitoring and reliability

- Monitor tab shows pipeline and activity runs; re-run from a failed activity.
- Per-activity **retry** count and interval; **timeout**.
- Alerts via Azure Monitor.
- **Idempotency matters**: a re-run must not double-load. Use overwrite semantics, a watermark, or a Delta `MERGE`.

## 3. Databricks

### Workspace concepts

- **Cluster** — the Spark compute. **All-purpose** clusters are interactive and shared; **job** clusters are created for a single run and terminated after, which is cheaper and better isolated.
- **Notebook** — cells in Python/SQL/Scala/R. `%python`, `%sql`, `%md`, `%sh`, `%run ./other_notebook`, `%pip install`.
- **Job / Workflow** — scheduled or triggered execution of notebooks/JARs/wheels with task dependencies.
- **DBFS** — the file system abstraction. Prefer mounting ADLS or, better, Unity Catalog external locations.
- **Unity Catalog** — centralised governance: three-level namespace `catalog.schema.table`, lineage, access control.
- **Repos** — Git integration so notebooks live in version control.

```
dbutils.fs.ls("/mnt/data")
dbutils.widgets.text("run_date", "")           # parameters from ADF/Jobs
dbutils.notebook.exit("success")               # return a value to the caller
dbutils.secrets.get(scope="kv", key="sql-pw")  # never print this
```

### Cluster sizing basics

Driver holds the plan and collected results; workers do the work. Autoscaling adjusts worker count within bounds. **Photon** is a vectorised C++ engine that accelerates SQL and DataFrame workloads. Spot/low-priority instances cut cost but can be reclaimed — fine for fault-tolerant batch, not for a run with a tight SLA.

### Delta Lake

```
MERGE INTO gold.sales t USING staging.sales s ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;

DESCRIBE HISTORY gold.sales;
SELECT * FROM gold.sales VERSION AS OF 12;      -- time travel
OPTIMIZE gold.sales ZORDER BY (customer_id);    -- compaction + data skipping
VACUUM gold.sales RETAIN 168 HOURS;
```

Delta gives ACID transactions, schema enforcement and evolution, time travel, and upsert. `OPTIMIZE` fixes the small-files problem; `ZORDER` co-locates related values so more files can be skipped.

### ADF + Databricks together

The standard pattern: ADF orchestrates (scheduling, dependencies, retries, alerting) and calls Databricks notebooks via the **Databricks Notebook activity**, passing parameters as widgets. Databricks does the compute. ADF handles the control plane; Databricks handles the data plane.

## 4. Azure DevOps & Code Industrialization

“Code industrialization” means moving from notebooks-on-someone’s-laptop to versioned, reviewed, tested, automatically deployed code.

### Repos and branching

**Git flow variants** you’ll meet: - `main` / `master` — production, protected. - `develop` — integration branch. - `feature/<ticket>-description` — one branch per unit of work. - `release/x.y`, `hotfix/...`

**Branch policies** on protected branches: require a pull request, minimum number of reviewers, linked work item, build must pass, no direct pushes.

**Pull request etiquette**: small and focused; a description explaining *why*, not just *what*; self-review before requesting others; respond to every comment; never merge your own PR where policy forbids it.

**Merge strategies**: merge commit (preserves history), squash (one tidy commit per PR — common), rebase (linear history, rewrites

commits).

```
git checkout -b feature/FEU-123-add-lag-features
git add -p && git commit -m "feat: add lag and rolling features to forecast prep"
git push -u origin feature/FEU-123-add-lag-features
git fetch origin && git rebase origin/develop
```

**Commit message convention** (Conventional Commits): `feat:`, `fix:`, `refactor:`, `docs:`, `test:`, `chore:`. Imperative mood, under ~72 characters in the subject.

## Repository structure

A defensible layout for a data/ML project:

```
project/
  src/           package code – importable, testable
    ingestion/
    transforms/
    features/
    models/
  notebooks/     exploration; thin, calling into src/
  tests/         unit and integration tests mirroring src/
  config/        environment configs (no secrets)
  pipelines/     ADF/Databricks job definitions as JSON/YAML
  docs/
  requirements.txt / pyproject.toml
  README.md
  .gitignore
```

**The principle: notebooks call functions; they don't contain the logic.** Logic in `src/` can be unit-tested, reviewed as a diff, and reused. Logic buried in a notebook cell cannot.

## CI/CD pipelines

```
trigger:
  branches: { include: [develop, main] }

pool: { vmImage: 'ubuntu-latest' }

steps:
  - task: UsePythonVersion@0
    inputs: { versionSpec: '3.11' }
  - script: pip install -r requirements.txt
    displayName: Install
  - script: ruff check src/ && black --check src/
    displayName: Lint
  - script: pytest tests/ --cov=src --cov-report=xml
    displayName: Test
  - task: PublishTestResults@2
```

**CI** = on every push/PR, build, lint and test automatically. **CD** = deploy the validated artefact to Dev → Test/UAT → Prod, with approval gates between environments. Infrastructure and pipeline definitions are stored as code (ARM/Bicep/Terraform, ADF JSON) so environments are reproducible.

**Environment promotion:** the same artefact moves through environments; only configuration changes. Never rebuild per environment — that defeats the purpose of testing the artefact.

## Coding standards

- **PEP 8** and a formatter ( `black` ) plus a linter ( `ruff` / `flake8` ) enforced in CI, so style is never a review topic.
- **Naming:** `snake_case` functions and variables, `PascalCase` classes, `UPPER_SNAKE` constants. Descriptive over short.
- **Docstrings** on every public function — purpose, args, returns, raises.
- **Type hints** on function signatures.
- **No hard-coded paths, credentials or magic numbers** — use config and Key Vault.
- **Logging, not print** . Use the `logging` module with levels; logs need timestamps and context to be useful in a job run.
- **Error handling:** catch specific exceptions, fail loudly on data-quality violations, and make failures diagnosable.
- **DRY**, small functions, single responsibility.
- **Unit tests** for transformation logic; `chispa` or comparing collected rows for Spark DataFrames.

```

import logging
logger = logging.getLogger(__name__)

def add_lag_features(df: DataFrame, cols: list[str], lags: list[int]) -> DataFrame:
    """Add lagged versions of the given columns, partitioned by SKU.

    Args:
        df: input frame, must contain 'sku' and 'date'.
        cols: columns to lag.
        lags: lag offsets in periods.
    Returns:
        DataFrame with one new column per (col, lag) pair.
    Raises:
        ValueError: if a required column is missing.
    """
    missing = {"sku", "date"} - set(df.columns)
    if missing:
        raise ValueError(f"missing required columns: {missing}")
    logger.info("adding %d lag features", len(cols) * len(lags))
    ...

```

## Data quality gates

Assert on the things that silently corrupt results: row-count preservation across a 1:1 join, primary-key uniqueness, null rates within expected bounds, referential integrity, and value ranges. Fail the job rather than writing bad data downstream. Tools: Great Expectations, [dbt](#) tests, Delta constraints, or plain asserts in the job.

## 5. Backtesting and Parallel Run

Week 4's operational topic. Both are about proving a change is safe before it takes over.

### Backtesting

Running the forecasting model over **historical** periods as if it were live, to estimate the accuracy you'd have achieved. Key requirements: - Use only information available at each historical decision point — **no lookahead**. - Use rolling-origin evaluation, not a single split. - Report the same KPIs the business uses (FA, bias) at the same aggregation level. - Compare against the incumbent model and against naive baselines.

**STP** in this context most likely means straight-through processing — the proportion of forecasts that flow through without manual intervention. Confirm the definition with the team; acronyms vary by organisation.

### Parallel run

Running the new system **alongside** the existing one on live data, both producing output, with only the incumbent's output used for decisions. You compare them over an agreed window before cutting over.

- Purpose: catch discrepancies under real conditions, including data issues that never appear in historical backtests.
- Exit criteria should be agreed **before** starting: what tolerance of difference is acceptable, over what period, on which metrics.
- Investigate every material discrepancy — differences are information, not noise to be averaged away.

This is the same idea as a **shadow deployment** in software: new code receives production traffic, its output is logged and compared, but it doesn't serve users.

### Run monitoring and support

Job success/failure alerting, SLA tracking (did the forecast land before the planning cycle needs it), data freshness checks, drift monitoring on inputs and on accuracy, runbooks for common failures, and an on-call/escalation path. In a support rotation the questions are always: is it failing, is it late, is it wrong — and each needs different instrumentation.

## 6. Questions Worth Being Able to Answer

- Difference between all-purpose and job clusters, and when each is appropriate
- Schedule trigger vs tumbling-window trigger
- Why medallion architecture, and what specifically goes in Silver
- What Delta gives you over plain Parquet
- Why notebooks should call [src/](#) code rather than contain logic
- What CI actually runs, and what blocks a merge
- How secrets reach a Databricks notebook without appearing in code
- What makes a pipeline idempotent, and why re-runs demand it
- Backtesting vs parallel run — what each catches that the other doesn't
- How you'd detect that a forecast job silently produced wrong numbers

## 7. What to Capture in FEU KT Sessions

Generic knowledge won't cover these. Take notes specifically on:

- ☐ Which Azure subscriptions/resource groups, and how environments are separated

- ☐ Storage layout and zone naming — does it follow medallion, and what are the actual container names
- ☐ Which orchestration tool actually triggers production runs, and on what schedule
- ☐ Cluster configurations and who can create them
- ☐ Repo structure, branch naming, and the exact PR policy
- ☐ The CI pipeline's stages and what causes a build to fail
- ☐ Deployment path from dev to prod, and who approves
- ☐ The backtesting harness — where it lives, how it's invoked, what it reports
- ☐ Parallel run exit criteria used historically
- ☐ Where run failures are alerted, and the escalation path